Uvod u softversko inženjerstvo
Avalonia UI framework

Fakultet tehničkih nauka
Univerzitet u Novom Sadu

Migration Status

> **Migration note:** This document started as a WPF PDF extraction (Phase 1). Tooling sections are now being rewritten for Avalonia + Rider (Phase 2). Figures are placeholders and will be replaced with Rider screenshots.

- **Done:** LaTeX scaffold + preserved original structure/text.
- **In progress:** Rider + Avalonia project creation/workflow sections.
- **Next:** convert WPF-specific APIs/examples to Avalonia equivalents; update screenshots; update docs links.

Contents

1 Uvod u Avalonia **2**
1.1 Kreiranje Avalonia projekta (Rider) . . . 2
1.2 Markup i code-behind . . . 3

2 Markup (XAML) **4**

3 Code-behind **5**

4 Događaji i obrada događaja (eng. Events and handling events) **6**
4.1 Dodavanje event handler-a u Avalonia (Rider) . . . 7

5 Paneli (eng. Panels) **8**
5.1 Wrap panel . . . 8
5.2 Stack panel . . . 9
5.3 Grid panel . . . 9
5.4 Dock panel . . . 10
5.5 Primer kombinacije više panela – Complex Layout . . . 10

6 Data binding **10**
6.1 Binding na resurs – string (1) . . . 11
6.2 Binding na resurs – boja (2) . . . 12
6.3 One way – ka interfejsu (7) . . . 12
6.4 Two way – bidirekciono (6) . . . 13
6.5 One way to source – ka kodu (8) . . . 13
6.6 Binding kontrole na kontrolu (3) . . . 13
6.7 Binding kontrole na kontrolu - real time (4) . . . 14
6.8 Binding kontrole na kontrolu - real time + konverzija (5) . . . 14
6.9 Binding na listu (9) . . . 14

1 Uvod u Avalonia

> **Migration note:** Phase 2: section is updated for Avalonia + Rider. Remaining sections still reflect WPF and will be migrated in later phases.

Avalonia UI je cross-platform GUI (Graphical User Interface) framework za kreiranje desktop aplikacija na Windows-u, Linux-u i macOS-u.

GUI framework omogućava da kreiramo aplikacije uz pomoć velikog opsega GUI elemenata kao što su labele (eng. labels) i polja za unos teksta (eng. textboxes). Bez GUI framework-a, morali bi da crtamo ove elemente ručno i da upravljamo sa svim korisničkim interakcijama, kao što je unos teksta ili pomeranje miša.

Avalonia sadrži set application-development alata (eng. features): XAML, kontrole, data binding, layouts, 2D grafika, animacije, stilovi, media i slično.

1.1 Kreiranje Avalonia projekta (Rider)

> **Migration note:** Phase 2: instructions are Rider-first with a CLI fallback, to be OS-agnostic.

Preduslovi:

- .NET SDK (8.0 ili noviji)
- JetBrains Rider
- (Opcionalno) Rider plugin *AvaloniaRider* za live XAML preview

Instalacija Avalonia templates:

```
dotnet new list
dotnet new install Avalonia.Templates
```

Kreiranje projekta u Rider-u:

1. Na početnom ekranu izaberite **New Solution**.
2. U sekciji **Custom Templates** izaberite **Avalonia .NET MVVM App** (preporučeno).
3. Unesite ime rešenja/projekta i lokaciju.
4. Kliknite **Create**.

Placeholder for figure file
`assets/slika-1.png`

Figure 1: Slika 1. Kreiranje novog Avalonia projekta u Rider-u

Alternativa (command line):

```
dotnet new avalonia.mvvm -o GetStartedApp
```

Pokretanje projekta:
U Rider-u kliknite **Run**. Alternativno, u terminalu pokrenite `dotnet run` iz desktop projekta (npr. `*.Desktop`).

Placeholder for figure file
`assets/slika-2.png`

Figure 2: Slika 2. Pokretanje Avalonia aplikacije (podrazumevani ekran)

1.2 Markup i code-behind

> **Migration note:** Phase 2: section is updated for Avalonia + Rider. Screenshots are placeholders and will be re-captured.

Avalonia omogućava da se razvija aplikacija koristeći markup i code-behind.
XAML markup se obično koristi za implementaciju izgleda aplikacije dok se code-behind koristi za implementaciju ponašanja tih komponenti.
Ovo razdvajanje na izgled i ponašanje povlači neke od prednosti kao što su:

1. Cena razvoja i održavanja aplikacije je smanjena jer je znatno lakše upravljati sa aplikacijom kod koje izgled i ponašanje nisu usko vezani (eng. tightly coupled).
2. Razvoj aplikacije je ubrzan i efikasan jer dizajneri mogu implementirati izgled aplikacije nezavisno i paralelno od programera koji implementiraju ponašanje aplikacije.
3. Globalizacija i lokalizacija za desktop aplikacije je pojednostavljena.

Na slici je prikazan prazan Avalonia projekat otvoren u Rider-u. Tipični segmenti rada su:

1. **Solution** prozor (pregled projekta i fajlova).
2. **XAML editor** (npr. `MainWindow.axaml`).
3. (Opcionalno) **XAML preview** uz Rider plugin *AvaloniaRider*.
4. **Code-behind** fajl (npr. `MainWindow.axaml.cs`).

Za razliku od tipičnog WPF+Visual Studio workflow-a, u Rider-u se kontrole najčešće dodaju pisanjem XAML-a uz code completion i snippet-e, a ne prevlačenjem iz Toolbox-a.
Otvaranjem `MainWindow.axaml.cs` dobija se code-behind.
Prelazak između XAML fajla i code-behind fajla vrši se iz **Solution** prozora ili korišćenjem pretrage (Search Everywhere).
Ako koristite XAML preview, obično je dostupan kao kartica **Preview** pored XAML editora (uz instaliran *AvaloniaRider* plugin).

Placeholder for figure file
`assets/slika-3.png`

Figure 3: Slika 3. Rider okruženje: Solution, XAML editor i (opciono) XAML preview

Placeholder for figure file
`assets/slika-4.png`

Figure 4: Slika 4. Code-behind (`*.axaml.cs`)

2 Markup (XAML)

> **Migration note:** Phase 2: section is updated for Avalonia. Examples use `.axaml` and Avalonia XAML namespaces.

XAML je XML-bazirani markup jezik koji omogućava implementiranje izgleda aplikacije (eng. application's appearance). Koristi se za kreiranje prozora, jednostavnih dijaloga i kontrola a navedeno dalje možemo popunjavati sa novim kontrolama, oblicima i grafičkim elementima.

U Avalonia projektima XAML fajlovi obično imaju ekstenziju `.axaml` (npr. `MainWindow.axaml`).

Primer prikazuje Avalonia XAML kod jednostavnog prozora (tag `<Window>`) koji sadrži samo jedno dugme (tag `<Button>`).

```
<Window xmlns="https://github.com/avaloniaui"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        x:Class="GetStartedApp.Views.MainWindow"
        Title="MainWindow" Width="800" Height="450">
  <Button Content="Click me" />
</Window>
```

Svaki element može da sadrži određene atribute, slično kao u XML-u. U primeru se vidi da tag `<Window>` sadrži Title atribut koji je zapravo tekst u title baru.

U primeru je prikazano kako se uz pomoć FontWeight property može postaviti boldovan tekst sa sadržajem "A button".

Još jedan način definisanja atributa je prikazan. Ovaj način prikazuje atribute kao child tagove glavnog taga, a u ovom slučaju to je `<Button>`.

S obzirom da je XAML XML-bazirani jezik, UI koji se kreira predstavlja hijerarhiju ugnježdenih elementa poznatih pod nazivom element tree.

Dat je primer gde je u okviru jednog dugmeta kreirano više tekst blokova koristeći više `<TextBlock>` tagova u `<Button>` tagu.

Placeholder for figure file
`assets/slika-5.png`

Figure 5: Slika 5. Navigacija: otvaranje `MainWindow.axaml.cs` iz Solution prozora

Placeholder for figure file
`assets/slika-6.png`

Figure 6: Slika 6. XAML preview u Rider-u (opciono)

Tekst u tekst blokovima je obojen različitim bojama koristeći atribut Foreground.

Content property dozvoljava samo jedan child element pa je bilo potrebno obuhvatiti child komeponente u WrapPanel. Paneli, kao što je WrapPanel, igraju veoma važnu ulogu u Avalonia i predstavljaju kontejnere za druge kontrole.

Isti rezultat se može postići na sledeći način. Ovaj način je jednostavniji i češće je u upotrebi.

3 Code-behind

> **Migration note:** Phase 2: section is updated for Avalonia. Example uses Avalonia event args and avoids `System.Windows.*`.

Osnovna odlika svake aplikacije je da implementira funkcionalnosti, uključujući obradu događaja (eng. handling events) kao što je korisnički klik. U Avalonia, ponašanje nakon okidanja događaja se može implementirati u kodu koji je povezan sa markup-om (XAML). Ovaj tip koda se naziva code-behind.

Primer prikazuje XAML kod sa event handler-om, a zatim code-behind koji odgovara tom kodu.

```
<!-- MainWindow.axaml -->
<Button Content="Klikni me" Click="Button_Click" />

// MainWindow.axaml.cs
using Avalonia.Controls;
using Avalonia.Interactivity;

public partial class MainWindow : Window
{
    public MainWindow()
    {
```

Placeholder for figure file
`assets/slika-7.png`

Figure 7: Slika 7. Jednostavan prozor sa jednim dugmetom (Avalonia XAML)

Placeholder for figure file
`assets/slika-8.png`

Figure 8: Slika 8. Atributi dugmeta - inline

```
        InitializeComponent();
    }

    private void Button_Click(object? sender, RoutedEventArgs e)
    {
        Title = "Kliknuto!";
    }
}
```

InitializeComponent metoda se poziva iz code-behind-a, iz konstruktora klase. Ova metoda zadužena je da spoji (eng. merge) UI koji se definiše u markup kodu sa code-behind klasom.

Napomena: InitializeComponent se generiše za vas kada se kreira aplikacija i ne treba je implementirati ručno.

4 Događaji i obrada događaja (eng. Events and handling events)

> **Migration note:** Phase 2: section is updated for Avalonia + Rider. Replaces Visual Studio Properties/Designer workflow with XAML-first editing.

Većina UI framework-a su event driven. Sve kontrole, uključujući `Window` kontrolu, izlažu događaje na koje se može odreagovati. Možete se prijaviti (eng. subscribe) na ove događaje, što znači da će vaša aplikacija biti obaveštena kada se ovaj događaj desi i tada je moguće odreagovati na određeni način. Reakcija na događaj se zapravo definiše u event handler-u.

Postoji više tipova događaja ali oni koji se najčešće koriste su vezani za interakciju sa mišem ili tastaturom. U Avalonia se često koriste događaji kao što su: `KeyDown`, `KeyUp`, `PointerPressed`, `PointerReleased`, `PointerEntered`, `PointerExited`.

Placeholder for figure file
`assets/slika-9.png`

Figure 9: Slika 9. Atributi dugmeta - child komponente

Placeholder for figure file
`assets/slika-10.png`

Figure 10: Slika 10. Složeni oblik dugmeta - prvi način

4.1 Dodavanje event handler-a u Avalonia (Rider)

1) Direktno u XAML-u

U XAML-u dodajte handler kao atribut kontrole (npr. dugmeta). Ako metoda ne postoji, Rider obično nudi quick-fix (**Alt+Enter**) da se kreira metoda u odgovarajućem code-behind fajlu.

```
<!-- MainWindow.axaml -->
<Button Content="Klik" Click="Button_Click" />

// MainWindow.axaml.cs
using Avalonia.Interactivity;

private void Button_Click(object? sender, RoutedEventArgs e)
{
    // reakcija na klik
}
```

2) Pretplata u code-behind-u

Drugi način je da se pretplatite na događaj u konstruktoru (posle `InitializeComponent()`). U tom slučaju najčešće je potrebno pronaći kontrolu po imenu (`Name` / `x:Name`).

```
// MainWindow.axaml
<Button x:Name="MyButton" Content="Klik" />

// MainWindow.axaml.cs
using Avalonia.Controls;
using Avalonia.Interactivity;

public MainWindow()
```

Placeholder for figure file
`assets/slika-11.png`

Figure 11: Slika 11. Složeni oblik dugmeta - drugi način

Placeholder for figure file
`assets/slika-12.png`

Figure 12: Slika 12. Proširen markup kod sa event handler-om

```
{
    InitializeComponent();
    var button = this.FindControl<Button>("MyButton");
    button.Click += Button_Click;
}
```

Napomena: Avalonia nema WPF-ov "Properties" panel za događaje na isti način kao Visual Studio designer; u praksi se handler-i dodaju kroz XAML ili kod.

5 Paneli (eng. Panels)

> **Migration note:** Phase 2: panels section is largely identical between WPF and Avalonia; text is kept with minor wording updates.

Panel, kao jedna od najvažnijih kontrola, predstavlja kontejner za druge kontrole i upravlja sa rasporedom istih na prozoru. S obzirom da Window kontrola može da ima samo jednu child kontrolu, obično se koristi panel da podeli prostor na regije gde svaka regija može ponovo biti panel.

U nastavku će biti navedeni i objašnjeni neki od panela koji se koriste u Avalonia.

5.1 Wrap panel

WrapPanel pozicionira svaku od svojih child kontrola jednu do druge, horizontalno (default varijanta) ili vertikalno, dokle god ima dovoljno prostora. Kada ne postoji dovoljno prostora, child elementi se wrap-uju u novi red i nastavlja se njihovo ređanje.

WrapPanel treba koristiti kada imamo listu kontrola koje želimo da rasporedimo vertikalno ili horizontalno.

Kada WrapPanel koristi horizontalnu orijentaciju, child kontrole će imati istu visinu, zasnovanu na visini najvišeg elementa. Kada je WrapPanel vertikalno orijentisan, child kontrole će imati istu širinu kao najširi element.

Placeholder for figure file
`assets/slika-13.png`

Figure 13: Slika 13. Code behind sa event handler-om

Placeholder for figure file
`assets/slika-14.png`

Figure 14: Slika 14. Dodavanje događaja kao XAML atribut (Rider)

Na slici je dat primer rasporeda elemenata u okviru Wrap panela.

5.2 Stack panel

StackPanel je veoma sličan Wrap panelu ali sa jednom važnom razlikom, ne wrap-uje sadržaj. Omogućava da se elementi ređaju jedan na drugi, u određenom smeru.

Treba primetiti da je default ponašanje Stack panela da razvlači (eng. Stretch) svoje chid kontrole.

StackPanel koji ima vertikalnu orijentaciju, sve svoje child kontrole rasteže horizontalno. Kada je u pitanju StackPanel sa horizontalnom orijentacijom, sve child kontrole se rastežu vertikalno.

StackPanel omogućava to zato što su HorizontalAlignment ili VerticalAlignment property na njegovim child kontrolama postavljeni na Stretch. Ovo ponašanje se može promeniti.

Takođe treba uočiti da je default orijentacija za StackPanel Vertical, za razliku od WrapPanel kod kog je Horizontal.

5.3 Grid panel

Grid može da sadrži više redova i kolona. Moguće je definisati visinu za svaki od redova ili širinu za svaku kolonu. Visina ili širina se mogu definisati na više načina, kao broj piksela, procentima u zavisnosti od slobodnog prostora ili da se vrednost prilagodi veličini sadržaja.

Uz pomoć RowDefinition atributa odredili smo da će Grid panel imati 3 reda. Za prva dva reda visina je postavljena na Auto.

Kako smo odredili koliko će Grid imati redova, možemo da odredimo i koliko kolona želimo u Grid-u. Ovo ćemo uraditi uz pomoć ColumnDefinition atributa. Za prve dve kolone širina je postavljena na Auto dok je za treću uzeta default vrednost.

Default vrednost za širinu je GridLength i predstavlja "1*" veličinu.

U XAML kodu možemo primetiti da je dodato 6 dugmića i da je za svaki definisano u kojoj koloni i redu se nalazi. Ovakvo ponašanje omogućavaju Grid.Column i Grid.Row atributi.

Placeholder for figure file
`assets/slika-15.png`

Figure 15: Slika 15. Pretplata na događaj u code-behind-u (Rider)

Placeholder for figure file
`assets/slika-20.png`

Figure 16: Slika 20. PrimerVezbe3 - WrapPanel

Osim toga, dodati su atributi koji određuju boju pozadine, kao i ColumnSpan i RowSpan.

5.4 Dock panel

DockPanel olakšava raspoređivanje sadržaja u 4 smera (top, bottom, left, right). Predstavlja najbolji izbor kada je potrebno podeliti prozor u specifične sekcije, naročito zbog toga što poslednji element u DockPanelu, po default-u, zauzima preostali prostor (center). Na taj način se sav prostor u prozoru zauzima.

DockPanel.Dock property omogućava da odredimo smer gde će kontrola biti postavljena.

Dugmići u drugom panelu su raspoređeni u različitim smerovima a dugme 'CETIRI' osim toga zauzima i centralni deo jer je poslednji naveden.

5.5 Primer kombinacije više panela – Complex Layout

U primeru je iskorišćen DockPanel da bi se prostor podelio na sekcije.

Na vrhu (DockPanel.Dock = "Top") postavljen je jedan TextBlock. Ista stvar urađena je i na dnu prozora.

Ostatak prostora popunjen je sa novim DockPanelom u okviru kog je kreiran StackPanel i dodat jedan TextBlock. StackPanel u sebi objedinjuje dva dugmeta a svaki od njih određen je atributima kao što su Height, Width i Margin.

Ovim primerom pokazano je kako se kompozicijom više panela može napraviti željeni raspored komponenti.

6 Data binding

> **Migration note:** Phase 2: section is updated for Avalonia. Examples still use placeholders; they will be aligned to the new course project in a later step.

Placeholder for figure file
`assets/slika-21.png`

Figure 17: Slika 21. PrimerVezbe3 – Izgled WrapPanel-a nakon pokretanja aplikacije

Placeholder for figure file
`assets/slika-22.png`

Figure 18: Slika 22. PrimerVezbe3 – StackPanel

Većina aplikacija omogućava korisniku da manipuliše sa nekim podacima. Kada su podaci učitani u aplikaciju (npr. iz neke baze podataka) potrebno ih je prikazati korisniku. Osim toga, korisniku se često omogućava da menja te podatke pa se mora voditi računa da bude tačno stanje podataka u bazi.

Da bi olakšala prikaz i interakciju sa podacima, Avalonia koristi data binding. Data binding predstavlja mehanizam koji omogućava komunikaciju UI elemenata sa podacima. Kada je binding uspostavljen i kada se podaci promene, moguće je automatski promeniti i UI elemente koji zavise od tih podataka. Obrnuto je takođe moguće.

Moguće je vršiti binding između dve kontrole kao i binding na resurs.

Da bi se mogle detektovati promene na source-u (primenjivo na OneWay i TwoWay binding), source mora implementirati odgovarajući mehanizam notifikacije promena, kao što je `INotifyPropertyChanged`.

Na primeru biće objašnjeni različiti načini upotrebe binding-a.

6.1 Binding na resurs – string (1)

Na slici je prikazano kako se može dodati resurs. U ovom slučaju, resursi su vezani za prozor tako da ih mogu koristiti sve kontrole iz tog prozora. Uz pomoć atributa x:Key se jedinstveno identifikuje i na taj način će mu se pristupati iz ostatka koda.

Ako želimo da TextBox kao svoj sadržaj ima tekst iz resursa, možemo to uraditi tako što ćemo Text property ove kontrole bindovati na resurs – string.

Sa ključnom reči StaticResource označavamo da će se koristiti statički resurs a kao vrednost ResourceKey navodimo jedinstveni identifikator resursa kog želimo da koristimo.

S obzirom da smo definisali vrednost x:Key kao “strTestBinding”, na ovaj način ćemo pozivati resurs.

Placeholder for figure file
`assets/slika-23.png`

Figure 19: Slika 23. PrimerVezbe3 - Izgled StackPanel-a nakon pokretanja aplikacije

Placeholder for figure file
`assets/slika-24.png`

Figure 20: Slika 24. PrimerVezbe3 - GridPanel

6.2 Binding na resurs – boja (2)

Na slici možemo videti još jedan resurs. Ovaj resurs definisan je kao SolidColorBrush i koristiće se kao boja za selektovani tekst.

Ovom resursu smo dodelili jedinstveni identifikator “clrSpecialColor”.

TextBox nudi SelectionBrush i na ovom mestu možemo odrediti boju koja se koristi kada se selektuje tekst iz ovog TextBox-a. Tako je na ovom mestu odabran resurs i bindovan je na SelectionBrush.

6.3 One way – ka interfejsu (7)

OneWay binding – promene na source property (model) automatski osvežavaju target property (UI kontrola) ali promene na target property (UI kontrola) ne emituju svoje promene na source property (model).

Na kratko ćemo zanemariti UpdateSourceTrigger, objašnjenje sledi u nastavku.

U ovom primeru biće navedeno kako bindovati “Test1” na Text property TextBox kontrole. “Test1” se nalazi u klasi SimpleBinding.xaml.cs.

Da bi mogli da izvršimo binding, property iz code-behind-a sa nekom kontrolom (u ovom slučaju TextBox) u XAML kodu, potrebno je da popunimo DataContext.

DataContext property je default source za naš binding, osim ako se ne specificira drugačije.

Ne postoji default source za DataContext property (on je null na početku), ali s obzirom da se DataContext nasleđuje dalje u hijerarhiji kontrola, često je dovoljno postaviti DataContext na ceo prozor i dalje koristiti za sve child kontrole.

Nakon poziva InitalizeComponent() metode, postavljamo "this" referencu kao DataContext, što u suštini govori Window komponenti da želimo da on sam bude Data context u bindingu.

Napomena: Obratiti pažnju na način kako funkcioniše!

Placeholder for figure file
`assets/slika-25.png`

Figure 21: Slika 25. PrimerVezbe3 - Izgled GridPanel-a nakon pokretanja aplikacije

Placeholder for figure file
`assets/slika-26.png`

Figure 22: Slika 26. PrimerVezbe3 – DockPanel

6.4 Two way – bidirekciono (6)

TwoWay binding – promene na source property (model) ili target property (UI kontrola) automatski osvežavaju promene na drugom. Ovaj tip binding-a odgovara npr. situaciji kada se koriste forme za editovanje podataka.

Na kratko ćemo zanemariti UpdateSourceTrigger, biće objašnjeno u nastavku.

Napomena: Obratiti pažnju na način kako funkcioniše!

6.5 One way to source – ka kodu (8)

OneWayToSource binding – suprotno od OneWay binding-a. Osvežava source property (model) kada dođe do promene na target property (UI element).

Napomena: Obratiti pažnju na način kako funkcioniše!

6.6 Binding kontrole na kontrolu (3)

Osim binding-a na resurs ili na property iz code-behind-a, moguće je bindovati kontrolu na kontrolu. Ovaj proces podrazumeva da ponašanje jedne kontrole zavisi od ponašanja druge kontrole.

U primeru se vidi kako vrednost TextBox kontrole zavisi od vrednosti Slider kontrole. Postiže se tako što Text property TextBox-a bindujemo na Value property Slider-a.

Generička sintaksa za ovakav binding u Avalonia može biti:

{Binding Value, ElementName=mySlider}

ili kraće, koristeći `#` sintaksu za named kontrolu:

{Binding #mySlider.Value}

Placeholder for figure file
`assets/slika-27.png`

Figure 23: Slika 27. PrimerVezbe3 - Izgled DockPanel-a nakon pokretanja aplikacije

Placeholder for figure file
`assets/slika-28.png`

Figure 24: Slika 28. PrimerVezbe3 - ComplexLayout

6.7 Binding kontrole na kontrolu - real time (4)

Binding kao što su TwoWay ili OneWayToSource osluškuju promene na target property (UI element) i propagiraju do source (model) i ovo je poznato kao "updating the source". Npr. kada se edituje polje (TextBox) treba promeniti source vrednost.

`UpdateSourceTrigger` određuje kada će se promena sa kontrole preneti na source. U Avalonia, `TextBox.Text` se podrazumevano sinhronizuje na svako kucanje (`PropertyChanged`). Ako želite da se source ažurira tek kada kontrola izgubi fokus, koristite `UpdateSourceTrigger=LostFocus`.

6.8 Binding kontrole na kontrolu - real time + konverzija (5)

Ako se u procesu binding-a želi iskoristiti konverzija, to se može lako uvesti uz pomoć Converter atributa. Dovoljno je navesti koji konverter će se koristiti. Da bi se kreirao konverter dovoljno je implementirati interface i implementirati telo Convert i ConvertBack metode.

Napomena: Obavezno pogledati primer u kodu!

6.9 Binding na listu (9)

U ovom primeru biće navedeno kako bindovati ComboBox vrednosti na vrednosti neke liste.

'Test3' predstavlja ObservableCollection u klasi SimpleBinding i sadržaće niz string vrednosti koje želimo prikazati u ComboBox kontroli.

Da bi mogli da povežemo ovu kolekciju iz SimpleBinding klase (code-behind) sa nekom kontrolom u XAML kodu, potrebno je da popunimo DataContext.

Upotreba DataContext objašnjena je u primeru sa One-way binding-om.

Napomena: Obavezno pogledati primer u kodu!

Klikom na dugme poziva se metoda u kojoj se vrši dodavanje elemenata u kolekciju.

Napomena: Obavezno pogledati da li će se promena manifestovati na ComboBox!

Placeholder for figure file
`assets/slika-29.png`

Figure 25: Slika 29. PrimerVezbe3 - Izgled ComplexLayout-a nakon pokretanja aplikacije

Placeholder for figure file
`assets/slika-30.png`

Figure 26: Slika 30. PrimerVezbe3 – Binding

DODATAK

Migration note: Rewrite appendix to Avalonia control list and links.

Kontrole (Avalonia reference):

- https://docs.avaloniaui.net/docs/reference/controls/

Često korišćeni layout-i (concept):

- **Canvas:** Child controls provide their own layout.
- **DockPanel:** Child controls are aligned to the edges of the panel.
- **Grid:** Child controls are positioned by rows and columns.
- **StackPanel:** Child controls are stacked either vertically or horizontally.
- **WrapPanel:** Child controls are positioned in left-to-right order and wrapped to the next line when there are more controls on the current line than space allows.

Korisni linkovi:

- https://docs.avaloniaui.net/docs/get-started/
- https://docs.avaloniaui.net/docs/get-started/xaml-previewers
- https://docs.avaloniaui.net/docs/get-started/wpf/
- https://docs.avaloniaui.net/docs/basics/data/data-binding/
- https://docs.avaloniaui.net/docs/basics/data/data-binding/data-binding-syntax
- https://docs.avaloniaui.net/docs/guides/data-binding/binding-to-controls
- https://play.avaloniaui.net/

Placeholder for figure file
`assets/slika-31.png`

Figure 27: Slika 31. PrimerVezbe3 – Resursi zakačeni na Window tag

Placeholder for figure file
`assets/slika-32.png`

Figure 28: Slika 32. PrimerVezbe3 – Binding na string

Placeholder for figure file
`assets/slika-33.png`

Figure 29: Slika 33. PrimerVezbe3 – Binding na boju

Placeholder for figure file
`assets/slika-34.png`

Figure 30: Slika 34. PrimerVezbe3 – Binding na boju - Rezultat selektovanja teksta

Placeholder for figure file
`assets/slika-35.png`

Figure 31: Slika 35. PrimerVezbe3 – One way binding

Placeholder for figure file
`assets/slika-36.png`

Figure 32: Slika 36. PrimerVezbe3 – Code behind – Test1 property

Placeholder for figure file
`assets/slika-37.png`

Figure 33: Slika 37. PrimerVezbe3 – Code behind – Konstruktor

Placeholder for figure file
`assets/slika-38.png`

Figure 34: Slika 38. PrimerVezbe3 – Two way binding

Placeholder for figure file
`assets/slika-39.png`

Figure 35: Slika 39. PrimerVezbe3 – One way to source binding

Placeholder for figure file
`assets/slika-40.png`

Figure 36: Slika 40. PrimerVezbe3 – Binding kontrole na kontrolu

Placeholder for figure file
`assets/slika-41.png`

Figure 37: Slika 41. PrimerVezbe3 – Binding kontrole na kontrolu – Prikaz nakon pokretanja aplikacije

Placeholder for figure file
`assets/slika-42.png`

Figure 38: Slika 42. PrimerVezbe3 - Real time binding kontrole na kontrolu

Placeholder for figure file
`assets/slika-43.png`

Figure 39: Slika 43. PrimerVezbe3 - Real time binding kontrole na kontrolu - Prikaz nakon pokretanja aplikacije

Placeholder for figure file
`assets/slika-44.png`

Figure 40: Slika 44. PrimerVezbe3 - Real time binding kontrole na kontrolu i konverzija

Placeholder for figure file
`assets/slika-45.png`

Figure 41: Slika 45. PrimerVezbe3 - Real time binding kontrole na kontrolu i konverzija - Prikaz nakon pokretanja aplikacije

Placeholder for figure file
`assets/slika-46.png`

Figure 42: Slika 46. PrimerVezbe3 – Konverter klasa

Placeholder for figure file
`assets/slika-47.png`

Figure 43: Slika 47. PrimerVezbe3 – Binding na listu vrednosti

Placeholder for figure file
`assets/slika-48.png`

Figure 44: Slika 48. PrimerVezbe3 – Code behind – Kolekcija